

- **Transactional paradigm:** ZFS being a transactional filesystem is guaranteed to be consistent according to its developers. Data management in ZFS uses copy on write semantics, which ensure that data is never overwritten, always maintaining an old reference to the data. A sequence of filesystem operations is either committed or ignored as a whole, thereby preventing any corruption to the filesystem due to power shortage or some other outage. This, in effect, removes the need for the *fsck* tool, the traditional filesystem check and repair tool.

- **Scrubbing and self-healing:** Since data and even metadata is checksummed, data scrubbing (an operation that checks data integrity within a filesystem or, in other words, data validation) is performed easily within ZFS. Checksum algorithms can be any user-selected algorithm from SHA-256 to Fletcher2, producing 256-bit long checksums. Besides checking for data integrity and preventing silent corruption, ZFS also provides mechanisms for self healing, mainly through RAID-Z and mirroring. Two RAID-Z variations, single and double-parity, are in fact slight variations of RAID-5 and RAID-6, respectively. The variations mainly aim to eliminate the write hole, solidifying data integrity. Besides, techniques like resilvering or resyncing help in replacing a corrupted or faulty device with a new one.

- **Scalability:** The team behind ZFS made the decision to go for a 128-bit filesystem, even though 64-bit filesystems like ext4 have come up only recently. Its data limit is an enormous 256 quadrillion zettabytes of storage which, is almost an impossible limit to reach in the near future since fully populating a 128-bit storage pool would, literally, require more energy than boiling the oceans, as Bonwick pointed out. Directories can have up to 2^{68} (256 trillion)

entries. No limit exists on the number of filesystems or number of files that can be contained within a filesystem.

- **Snapshots and clones:** Snapshot is a read-only copy of a filesystem or volume at any particular point of time. Its design is such that space is consumed only when data is changed, preventing any freeing of data from the filesystem unless explicitly asked, giving further options for maintaining data integrity. Clone is a writable filesystem generated from a snapshot. The creation of snapshots and clones in ZFS is very simple and is always pointed out as one of its big advantages.

ZFS and Linux

ZFS is the standard filesystem for Solaris/OpenSolaris OS whose source code is published under CDDL (Common Development and Distribution License). However, from the beginning (and hopefully forever) the Linux kernel has remained licensed under the GPLv2, which prevents any other code to be linked with the GPLv2 Linux kernel unless that code's licence is GPL v2 compatible. So the open sourced code of ZFS cannot be added/linked to the kernel code like any other filesystem, either as a part of the kernel or as kernel modules. As a workaround, some solutions pointed out by the open source community are:

1. A 'court ruling' (either in the US or EU, where ZFS is mainly used) stating that GPL and CDDL are compatible.
2. Either of the parties (Linux and Solaris) need to change the licence of their code to a mutually compatible one.
3. A GPLv2 ZFS reimplement from scratch, which should be free from all the 56 patents that Sun has taken on ZFS code.
4. A method by which we would be able to implement ZFS to be usable for Linux, which is only possible through dynamic linking between the codes—this is allowed.

NACE™
move to the next level

TRAINING & CONSULTANCY

There is a
Great Demand
for
NACE Certified
LAMP Professionals

**LINUX
APACHE
MySQL
PHP**



with *HTML, CSS
JAVA SCRIPT & AJAX*
**DRUPAL
NCLP 10**

CHENNAI

T.NAGAR

99405 42222

ANNA NAGAR

97909 25554

ADYAR

9790925550

www.naceedu.com